

CanlıFal — Flutter Resmî Servis Entegrasyon Dokümanı

Amaç: Flutter mobil uygulamasının, web sürümüyle **aynı veritabanını, aynı iş kurallarını ve aynı gerçek zamanlı davranışı** kullanmasını garanti etmek. Backend, Flutter için **resmî (birinci sınıf) servis** olarak tanımlanmıştır. Hiçbir özelliğe web ↔ mobil farkı kalmamalıdır.

Backend teknolojisi: Next.js App Router (`app/api/**/route.ts`) · Tek PostgreSQL (Prisma) · Paylaşımlı veritabanı (web + mobil aynı DB).

Base URL (prod): `https://canlifal.com`

Toplam endpoint: 731 route · 193 veri modeli.

Referans dokümanlar (aynı klasör / backend-docs/): `ENDPOINTS.md` (tüm endpoint listesi), `DATA-BASE_REFERENCE.md` (tüm modeller), `openapi.json`, `postman_collection.json`, `schema.prisma`.

Bu doküman, o referansların üzerine Flutter tarafının ihtiyaç duyduğu **13 konuyu** ekler:

Dart model şemaları · DTO yapıları · Enum listeleri · JSON örnekleri · API versiyonlama · Upload API'leri · CDN yapısı · Cloudflare/S3 dosya yolları · Cache kuralları · Offline senaryoları · Retry mekanizmaları · Background/Foreground davranışları · Acceptance Test senaryoları.

1. Temel İlkeler — Web/Mobil Parite

- Tek kaynak = Backend.** İş kuralları (komisyon oranları, jeton düşümü, oda kapasitesi, hediye dağıtımı) **yalnızca** backend'te hesaplanır. Flutter bu değerleri asla yeniden hesaplamaz; backend'in döndürdüğü sonucu gösterir.
- Aynı endpoint, aynı DB.** Mobil, web ile birebir aynı `/api/**` route'larını kullanır. Aynı bir "mobil DB" veya ayrı iş mantığı yoktur.
- Dinamik yapılandırma.** Görsel/animasyon/asset seçimleri, özellik bayrakları, sürüm zorlaması ve fiyatlar backend'ten çekilir (bkz. `/api/mobile/config`, `/api/gifts/catalog`). Uygulama içinde hard-code edilmez.
- Client-side render, server-side truth.** Animasyon/asset türünü server bildirir (`assetType`, `displayType`), oynatıcıyı client seçer; fakat parasal/durum sonucu her zaman server'dan gelir.
- Gerçek zamanlılık.** Web SSE (Server-Sent Events) kullanır; Flutter da aynı SSE stream'lerini tüketir (bkz. Realtime dokümanı). Ek olarak OneSignal push kullanılır.

2. API Versiyonlama

Backend **yol tabanlı sürüm (path-versioning) kullanmaz** — tüm route'lar `/api/...` altındadır ve **geriye dönük uyumlu** evrimleşir. Sürüm yönetimi üç mekanizmayla yapılır:

2.1 Uygulama Sürümü / Zorunlu Güncelleme — `GET /api/mobile/config`

Query: `?platform=ios|android&version=1.2.3`

Döner: bakım modu, zorunlu/opsiyonel güncelleme, mağaza URL'leri ve **özellik bayrakları**. Flutter **her açılışta (cold start)** bunu çağırmalı ve:

- `needsForceUpdate == true` → engelleyici güncelleme ekranı göster.
- `maintenanceMode == true` → bakım ekranı göster.
- Feature flag'lere göre menü/sekme gizle-göster.

```
{
  "maintenanceMode": false,
  "maintenanceMessage": "Bakım çalışması yapılmaktadır...",
  "needsForceUpdate": false,
  "hasOptionalUpdate": true,
  "minVersion": "1.0.0",
  "latestVersion": "1.4.0",
  "storeUrl": "https://play.google.com/...",
  "features": {
    "liveStream": true, "chat": true, "shortVideos": true,
    "games": true, "stories": true, "aiFortune": true,
    "liveTeller": true, "pkBattle": true
  },
  "supportEmail": "destek@canlifal.com",
  "termsUrl": "https://canlifal.com/tr/yasal/kullanim-sartlari",
  "privacyUrl": "https://canlifal.com/tr/yasal/gizlilik-politikasi"
}
```

2.2 İçerik Sürümü (Delta Senkronizasyon) — GET /api/gifts/version

Hafif sürüm damgası döndürür; Flutter bunu lokalde saklar ve yalnızca değiştiğinde ağır katalogu çeker:

```
{ "giftVersion": 12, "themeVersion": 3, "giftCount": 84, "themeCount": 6,
  "timestamp": "2026-07-23T..." }
```

Akış: `version` çağır → lokal `giftVersion` < server ise `GET /api/gifts/catalog?sinceVersion=<lokal>` ile **sadece değişenleri** çek. (`Cache-Control: s-maxage=60, stale-while-revalidate=120` .)

Aynı `contentVersion` deseni tüm katalog benzeri kaynaklarda kullanılır (hediyeler, oda temaları, kozmetikler).

2.3 Kırıcı Değişiklik Politikası

- Alan **eklenir**, asla anlamı değiştirilerek yeniden kullanılmaz.
- Alan **silinmez**; kullanımdan kalkacaksa `deprecated` sayılır ama JSON'da kalır.
- Yeni enum değerleri **sona** eklenir. Flutter enum parse'ında **bilinmeyen değer** → `unknown` fallback zorunludur (aşağıda `_enumFrom` yardımcı fonksiyonuna bakın).

3. Kimlik Doğrulama (Dual Auth)

Backend **çift kimlik doğrulama** kullanır (`lib/mobile-auth.ts` → `authenticateRequest`): Web → NextAuth session cookie; Mobil → **Bearer JWT**. Her iki yol da aynı route'larda çalışır.

Token	Ömür	Amaç
accessToken	7 gün	Her isteğin Authorization: Bearer <token> başlığında
refreshToken	30 gün	Süresi dolan access token'ı yenilemek için

Login: POST /api/auth/mobile-login · Body { email, password } (email alanı kullanıcı adı da olabilir).

Yenileme: POST /api/auth/mobile-refresh · Body { refreshToken } → yeni { accessToken, refreshToken, user }. Geçersiz/expired refresh → **401** (kullanıcıyı login'e at).

Kayıt: POST /api/auth/mobile-register .

Sosyal: POST /api/auth/mobile-google · .../mobile-apple · .../mobile-tiktok .

Tüm mobil isteklerde header:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Login yanıtı (gerçek alanlar)

```
{
  "accessToken": "eyJ...",
  "refreshToken": "eyJ...",
  "user": {
    "id": "clx...", "email": "a@b.com", "name": "Ayşe", "username": "ayse",
    "role": "user", "image": "https://upload.wikimedia.org/wikipedia/commons/thumb/4/4f/Evann_Guessand_Cote_D%27Ivoire_v_Ecuador_14_June_2026-28_%28cropped%29.jpg/250px-Evann_Guessand_Cote_D%27Ivoire_v_Ecuador_14_June_2026-28_%28cropped%29.jpg",
    "credits": 120, "jetonBalance": 5400, "cfcBalance": 0,
    "membership": "free", "membershipExpiresAt": null,
    "preferredLanguage": "tr", "level": 3, "bio": null, "phone": null,
    "birthDate": null, "zodiacSign": "Terazi", "referralCode": "AYSE1234"
  }
}
```

4. Standart Yanıt Zarfı, Hata Formatı ve HTTP Kodları

Backend'te iki yanıt stili birlikte bulunur; Flutter DTO katmanı **ikisini de** tolere etmelidir:

- Sarmalanmış (yeni/mobil route'lar):** { "success": true, "data": {...} } / hata: { "success": false, "error": { "code": "UNAUTHORIZED", "message": "..." } }
- Düz (klasik route'lar):** başarı → nesnenin kendisi; hata → { "error": "Mesaj (Türkçe)" }

HTTP kodları: 200/201 başarı · 400 geçersiz gövde · 401 kimlik yok/expired · 403 yetki yok · 404 bulunamadı · 409 çakışma (idempotency) · 429 rate limit · 500 sunucu.

Flutter tarafı hata mesajı çıkarımı: error.message ?? error (string) ?? 'Bilinmeyen hata'. Hata metinleri backend'te **Türkçe** üretilir; olduğu gibi gösterilebilir.

5. Sayfalama (Pagination)

Liste endpoint'leri `?page=<1..>&limit=<..>` alır ve genelde şu meta ile döner:

```
{ "items": [ ... ], "streams": [ ... ], "pagination": { "page": 1, "limit": 30, "total": 240, "totalPages": 8, "hasMore": true } }
```

Not: Bazı route'lar hem `items` hem alana-özel anahtar (`streams` , `rooms` , `tellers`) döndürür. Flutter parse'ı **önce alana-özel, yoksa `items` , yoksa kök dizi** sırasını denemelidir. Varsayılan `limit=30` , üst sınır genelde `100` .

6. Enum Listeleri

Backend enum'ları DB'de **string** olarak tutulur (Prisma native enum yok). Flutter'da her biri `unknown` fallback'li enum olarak modellenmelidir.

Enum	Değerler	Kaynak
UserRole	user , falci , teller , moderator , finans , editor , yönetici , admin	route yetki kontrolü
Membership	free , silver , gold , platinum , vip	user.membership
LiveSessionStatus	pending , active , completed , cancelled , rejected	/api/room , /api/fortune-tellers/sessions
VideoStreamStatus	live , ended	/api/video-streams
GiftAssetType	image , video , lottie , svga , gif	GiftType.assetType
GiftDisplayType	standard , fullscreen , banner , combo	GiftType.displayType
GiftEventStatus	completed , refunded , cancelled	GiftEvent.status
Currency	jeton (birincil) , credit , cfc	cüzdan alanları
FortuneType	coffee , tarot , palm , dream , astrology , numerology , katina , angel , aura , love , yesno , birthchart	fal kartları
RoomType	free , normal , vip	sesli oda kapasitesi
TransactionType	gift_sent , gift_received , purchase , session_charge , session_earning , refund , agency_commission , admin_adjust	JetonTransaction.type
NotificationType	session_request , session_accepted , gift_received , follow , message , stream_live , system	Notification.type
SignalType	offer , answer , ice-candidate	WebRTC signaling

Kesin ve tam alan/enum listesi için `backend-docs/DATABASE_REFERENCE.md` ve `schema.prisma` kaynaktır. Yeni değerler sona eklenebileceğinden Flutter parse'ı bilinmeyen değeri `unknown` 'a düşürmelidir.

7. Dart Model / DTO Şemaları

7.1 Ortak altyapı (envelope, pagination, enum yardımcı)

```
// api_response.dart
class ApiResponse<T> {
  final bool success;
  final T? data;
  final String? errorCode;
  final String? errorMessage;
  const ApiResponse({required this.success, this.data, this.errorCode, this.errorMessage});

  factory ApiResponse.fromJson(Map<String, dynamic> json, T Function(Object? j)
  parse) {
    // Sarmalanmış stil
    if (json.containsKey('success')) {
      final ok = json['success'] == true;
      if (ok) return ApiResponse(success: true, data: parse(json['data']));
      final err = json['error'];
      if (err is Map) {
        return ApiResponse(success: false, errorCode: err['code']?.toString(), errorMessage: err['message']?.toString());
      }
      return ApiResponse(success: false, errorMessage: err?.toString() ?? json['error']?.toString());
    }
    // Düz hata stili
    if (json.containsKey('error')) {
      return ApiResponse(success: false, errorMessage: json['error']?.toString());
    }
    // Düz başarı stili (nesnenin kendisi)
    return ApiResponse(success: true, data: parse(json));
  }
}

class Paginated<T> {
  final List<T> items;
  final int page, limit, total, totalPages;
  final bool hasMore;
  const Paginated({required this.items, this.page = 1, this.limit = 30, this.total = 0, this.totalPages = 0, this.hasMore = false});

  factory Paginated.fromJson(Map<String, dynamic> json, T Function(Object?) parse, {List<String> listKeys = const ['items']}) {
    List raw = const [];
    for (final k in [...listKeys, 'items']) {
      if (json[k] is List) { raw = json[k]; break; }
    }
    final p = (json['pagination'] as Map?) ?? const {};
    return Paginated(
      items: raw.map((e) => parse(e)).toList(),
      page: (p['page'] ?? 1) as int,
      limit: (p['limit'] ?? 30) as int,
      total: (p['total'] ?? raw.length) as int,
      totalPages: (p['totalPages'] ?? 1) as int,
      hasMore: (p['hasMore'] ?? false) as bool,
    );
  }
}

// Bilinmeyen enum değeri -> unknown fallback
E _enumFrom<E>(List<E> values, String? raw, E fallback) {
  if (raw == null) return fallback;
  for (final v in values) {
    if (v.toString().split('.').last.toLowerCase() == raw.toLowerCase()) return v;
  }
}
```

```
}  
    return fallback;  
}
```

7.2 Enum'lar (unknown fallback zorunlu)

```
enum UserRole { user, falci, teller, moderator, finans, editor, yonetici, admin, unknown }  
enum Membership { free, silver, gold, platinum, vip, unknown }  
enum LiveSessionStatus { pending, active, completed, cancelled, rejected, unknown }  
enum VideoStreamStatus { live, ended, unknown }  
enum GiftAssetType { image, video, lottie, svga, gif, unknown }  
enum GiftDisplayType { standard, fullscreen, banner, combo, unknown }  
enum GiftEventStatus { completed, refunded, cancelled, unknown }  
enum Currency { jeton, credit, cfc, unknown }  
enum RoomType { free, normal, vip, unknown }
```

7.3 UserDto (auth + profil)

```
class UserDto {
    final String id, email;
    final String? name, username, image, bio, phone, zodiacSign, referralCode;
    final UserRole role;
    final Membership membership;
    final Int credits, jetonBalance, cfcBalance, level;
    final DateTime? membershipExpiresAt, birthDate;
    final String preferredLanguage;

    const UserDto({required this.id, required this.email, this.name, this.username,
        this.image, this.bio, this.phone, this.zodiacSign, this.referralCode,
        this.role = UserRole.user, this.membership = Membership.free,
        this.credits = 0, this.jetonBalance = 0, this.cfcBalance = 0, this.level = 0,
        this.membershipExpiresAt, this.birthDate, this.preferredLanguage = 'tr'});

    factory UserDto.fromJson(Map<String, dynamic> j) => UserDto(
        id: j['id'].toString(), email: (j['email'] ?? '').toString(),
        name: j['name']?.toString(), username: j['username']?.toString(),
        image: j['image']?.toString(), bio: j['bio']?.toString(), phone: j['phone']?.to-
String(),
        zodiacSign: j['zodiacSign']?.toString(), referralCode: j['referralCode']?.to-
String(),
        role: _enumFrom(UserRole.values, j['role']?.toString(), UserRole.user),
        membership: _enumFrom(Membership.values, j['membership']?.toString(), Member-
ship.free),
        credits: (j['credits'] ?? 0) as int, jetonBalance: (j['jetonBalance'] ?? 0) as
int,
        cfcBalance: (j['cfcBalance'] ?? 0) as int, level: (j['level'] ?? 0) as int,
        membershipExpiresAt: _date(j['membershipExpiresAt']), birthDate: _date(j['birthD-
ate']),
        preferredLanguage: (j['preferredLanguage'] ?? 'tr').toString());
}

DateTime? _date(Object? v) => v == null ? null : DateTime.tryParse(v.toString());

class AuthResponse {
    final String accessToken, refreshToken;
    final UserDto user;
    const AuthResponse({required this.accessToken, required this.refreshToken, required
this.user});
    factory AuthResponse.fromJson(Map<String, dynamic> j) => AuthResponse(
        accessToken: j['accessToken'].toString(), refreshToken: j['refreshToken'].to-
String(),
        user: UserDto.fromJson(j['user'] as Map<String, dynamic>));
}
```

7.4 GiftTypeDto & GiftEventDto (hediye sistemi)

```
class GiftTypeDto {
    final String id, name;
    final int priceJeton;
    final String? assetUrl, thumbnailUrl;
    final GiftAssetType assetType;
    final GiftDisplayType displayType;
    final bool comboEnabled, isLucky, isActive;
    final int contentVersion;

    const GiftTypeDto({required this.id, required this.name, this.priceJeton = 0,
        this.assetUrl, this.thumbnailUrl, this.assetType = GiftAssetType.image,
        this.displayType = GiftDisplayType.standard, this.comboEnabled = false,
        this.isLucky = false, this.isActive = true, this.contentVersion = 1});

    factory GiftTypeDto.fromJson(Map<String, dynamic> j) => GiftTypeDto(
        id: j['id'].toString(), name: (j['name'] ?? '').toString(),
        priceJeton: (j['priceJeton'] ?? j['price'] ?? 0) as int,
        assetUrl: j['assetUrl']?.toString(), thumbnailUrl: j['thumbnailUrl']?.toString(),
        assetType: _enumFrom(GiftAssetType.values, j['assetType']?.toString(), GiftAsset-
Type.image),
        displayType: _enumFrom(GiftDisplayType.values, j['displayType']?.toString(), Gift-
DisplayType.standard),
        comboEnabled: j['comboEnabled'] == true, isLucky: j['isLucky'] == true,
        isActive: j['isActive'] != false, contentVersion: (j['contentVersion'] ?? 1) as in
t);
}

class GiftEventDto {
    final String id, giftTypeId, senderId, receiverId;
    final int quantity, grossJeton, receiverNetJeton;
    final GiftEventStatus status;
    final DateTime createdAt;
    const GiftEventDto({required this.id, required this.giftTypeId, required this.sen-
derId,
        required this.receiverId, this.quantity = 1, this.grossJeton = 0, this.receiv-
erNetJeton = 0,
        this.status = GiftEventStatus.completed, required this.createdAt});
    factory GiftEventDto.fromJson(Map<String, dynamic> j) => GiftEventDto(
        id: j['id'].toString(), giftTypeId: j['giftTypeId'].toString(),
        senderId: j['senderId'].toString(), receiverId: j['receiverId'].toString(),
        quantity: (j['quantity'] ?? 1) as int, grossJeton: (j['grossJeton'] ?? 0) as int,
        receiverNetJeton: (j['receiverNetJeton'] ?? 0) as int,
        status: _enumFrom(GiftEventStatus.values, j['status']?.toString(), GiftEvent-
Status.completed),
        createdAt: _date(j['createdAt']) ?? DateTime.now());
}
```

7.5 LiveSessionDto & VideoStreamDto

```
class LiveSessionDto {
    final String id;
    final String? fortuneType, roomId;
    final LiveSessionStatus status;
    final int maxMinutes, minutesUsed, creditsPerMinute, creditsCharged;
    final Map<String, dynamic>? teller; // {id, displayName, avatar}
    final DateTime? startedAt, createdAt;
    const LiveSessionDto({required this.id, this.fortuneType, this.roomId,
        this.status = LiveSessionStatus.pending, this.maxMinutes = 0, this.minutesUsed =
    0,
        this.creditsPerMinute = 0, this.creditsCharged = 0, this.teller, this.startedAt, t
his.createdAt});
    factory LiveSessionDto.fromJson(Map<String, dynamic> j) => LiveSessionDto(
        id: j['id'].toString(), fortuneType: j['fortuneType']?.toString(), roomId: j['room
Id']?.toString(),
        status: _enumFrom(LiveSessionStatus.values, j['status']?.toString(), LiveSession-
Status.pending),
        maxMinutes: (j['maxMinutes'] ?? 0) as int, minutesUsed: (j['minutesUsed'] ?? 0)
as int,
        creditsPerMinute: (j['creditsPerMinute'] ?? 0) as int, creditsCharged: (j['cred-
itsCharged'] ?? 0) as int,
        teller: (j['teller'] as Map?)?.cast<String, dynamic>(),
        startedAt: _date(j['startedAt']), createdAt: _date(j['createdAt']));
}

class VideoStreamDto {
    final String streamId;
    final String? title, streamerName, thumbnailUrl, coverUrl, broadcasterId, hos-
tUserId;
    final bool isLive;
    final int viewers, watching, likeCount;
    const VideoStreamDto({required this.streamId, this.title, this.streamerName, this.th
umbnailUrl,
        this.coverUrl, this.broadcasterId, this.hostUserId, this.isLive = false,
        this.viewers = 0, this.watching = 0, this.likeCount = 0});
    factory VideoStreamDto.fromJson(Map<String, dynamic> j) => VideoStreamDto(
        streamId: (j['streamId'] ?? j['id']).toString(), title: j['title']?.toString(),
        streamerName: j['streamerName']?.toString(), thumbnailUrl: j['thumbnailUrl']?.to-
String(),
        coverUrl: j['coverUrl']?.toString(), broadcasterId: j['broadcasterId']?.to-
String(),
        hostUserId: j['hostUserId']?.toString(), isLive: j['isLive'] == true,
        viewers: (j['viewers'] ?? 0) as int, watching: (j['watching'] ?? 0) as int,
        likeCount: (j['likeCount'] ?? 0) as int);
}
```

7.6 MobileConfigDto & NotificationDto

```
class MobileConfigDto {
  final bool maintenanceMode, needsForceUpdate, hasOptionalUpdate;
  final String? maintenanceMessage, minVersion, latestVersion, storeUrl, supportEmail;
  final Map<String, bool> features;
  const MobileConfigDto({this.maintenanceMode = false, this.needsForceUpdate = false,
    this.hasOptionalUpdate = false, this.maintenanceMessage, this.minVersion, this.latestVersion,
    this.storeUrl, this.supportEmail, this.features = const {}});
  factory MobileConfigDto.fromJson(Map<String, dynamic> j) => MobileConfigDto(
    maintenanceMode: j['maintenanceMode'] == true, needsForceUpdate: j['needsForceUpdate'] == true,
    hasOptionalUpdate: j['hasOptionalUpdate'] == true, maintenanceMessage: j['maintenanceMessage']?.toString(),
    minVersion: j['minVersion']?.toString(), latestVersion: j['latestVersion']?.toString(),
    storeUrl: j['storeUrl']?.toString(), supportEmail: j['supportEmail']?.toString(),
    features: ((j['features'] as Map?) ?? {}).map((k, v) => MapEntry(k.toString(), v == true)));
}
```

7.7 Tüm 731 endpoint için model kapsamı — Kod Üretim Stratejisi

Yukarıdaki DTO'lar **kritik/çekirdek** modellerdir. Kalan tüm endpoint/model kapsamı **elle değil, üretime** sağlanır (hata payını sıfırlamak için):

1. Kaynak sözleşme = `backend-docs/openapi.json` (468 path şeması + 731 route indeksi `endpoints_index.json`).
2. Flutter'da `build_runner + json_serializable (@JsonSerializable())` ile DTO üretimi; veya doğrudan `openapi-generator-cli generate -g dart-dio -i openapi.json`.
3. Üretilen istemci, bu dokümandaki `ApiResponse / Paginated` sarmalayıcı ve `_enumFrom` fallback deseniyle sarılır.
4. Her yeni backend sürümünde `openapi.json` yeniden üretilir (`scripts_docs/generate_api_docs.py`), Flutter modelleri yeniden generate edilir → **kalıcı parite**.

8. JSON Örnekleri (kritik akışlar)

Hediye gönder — `POST /api/live/gift/send` (stream + sesli oda ortak):

```
// İstek
{ "context": "stream", "contextId": "strm_123", "giftTypeId": "gift_rose",
  "receiverId": "usr_555", "quantity": 5, "idempotencyKey": "a1b2-...-uuid" }
// Yanıt
{ "success": true, "data": {
  "giftEventId": "ge_789", "grossJeton": 500, "receiverNetJeton": 245,
  "siteCommission": 150, "roomOwnerNet": 105,
  "newBalance": { "jetonBalance": 4900 }, "status": "completed" } }
```

Hediye katalog (delta) — `GET /api/gifts/catalog?sinceVersion=8` :

```
{ "version": 12, "gifts": [
  { "id": "gift_rose", "name": "Gül", "priceJeton": 100, "assetType": "svga",
    "assetUrl": "https://i.etsystatic.com/28473016/r/il/2a5ced/3698516764/
il_fullxfull.3698516764_kex8.jpg", "displayType": "standard",
    "comboEnabled": true, "isLucky": false, "contentVersion": 12 } ] }
```

Presigned upload — POST /api/upload/presigned :

```
// İstek
{ "fileName": "avatar.jpg", "contentType": "image/jpeg", "isPublic": true }
// Yanıt
{ "uploadUrl": "https://<bucket>.s3.<region>.amazonaws.com/...?X-Amz-...",
  "cloud_storage_path": "<prefix>public/uploads/1737650000-avatar.jpg" }
```

9. Upload API'leri

İki ayrı depolama arka ucu vardır. **Doğru olanı kullanın:**

9.1 Genel medya (avatar, hediye görseli, fal fotoğrafı) → AWS S3

- **Adım 1 — Presigned al:** POST /api/upload/presigned (Bearer zorunlu) · Body { fileName, contentType, isPublic }. Yalnız image/* ve video/* kabul edilir.
- **Adım 2 — Doğrudan S3'e PUT:** Dönen uploadUrl 'e ham baytları PUT et. isPublic:true ise presigned imzası ContentDisposition: attachment içerir → PUT isteğine Content-Disposition: attachment header'ı eklenmezse **403** alınır. (Ayrıntı: lib/s3.ts.)
- **Adım 3 — Kaydet:** Dönen cloud_storage_path 'i ilgili kaynağa (profil, hediye vb.) gönder. Uygulama yalnızca cloud_storage_path saklar, yerel yol değil.
- **Private dosya okuma:** POST /api/upload/get-url · Body { cloud_storage_path, isPublic } → süreli (1 saat) imzalı url. İmzalı URL'ler **DB'ye kaydedilmez**, anlık üretilir.

9.2 Kısa videolar (Shorts) → Cloudflare R2

- **Presigned:** POST /api/short-videos/upload-url · Body { type: 'video'|'thumbnail', contentType?, ext? } → { key, uploadUrl, publicUrl, contentType, expiresIn }.
- İstemci uploadUrl 'e ham baytları PUT eder → sonra publicUrl 'i POST /api/short-videos/register 'a gönderir.
- Alternatif küçük dosya: POST /api/short-videos/upload (sunucu üzerinden).

9.3 Admin hediye asset yükleme

- POST /api/admin/gift-upload (admin/yonetici) — SVGA/Lottie/video hediye asset'leri için.

9.4 Boyut kuralları (istemci zorunlu)

- file.size ≤ 100MB → tek parça presigned PUT.
- > 100MB → multipart upload (S3 aksi halde EntityTooLarge döner). Shorts videoları için R2 tek-parça 5GB'a kadar destekler; yine de büyük dosyada arka planda yükleme + presigned kullanın.

10. CDN Yapısı ve Cloudflare / S3 Dosya Yolları

10.1 AWS S3 (ana medya) — `lib/s3.ts` , `lib/aws-config.ts`

Anahtar (key) deseni `AWS_FOLDER_PREFIX` (`<prefix>`) ile başlar:

```
<prefix>public/uploads/<timestamp>-<fileName>      # isPublic:true  (herkese açık)
<prefix>uploads/<timestamp>-<fileName>              # isPublic:false (imzalı erişim)
```

- **Public URL:** `https://<AWS_BUCKET_NAME>.s3.<AWS_REGION>.amazonaws.com/<cloud_storage_path>`
Key'in her segmenti URL-encode edilir (boşluk/özel karakter içeren isimler için).
- **Private URL:** presigned GET (1 saat TTL), `POST /api/upload/get-url` ile alınır.

10.2 Cloudflare R2 (shorts) — `lib/r2-storage.ts`

Endpoint: `https://<R2_ACCOUNT_ID>.r2.cloudflarestorage.com` · Bucket: `R2_BUCKET_NAME` (varsayılan `canlifal-shorts`).

Klasör yapısı:

```
shorts/videos/<uuid>.mp4      (veya .mov)
shorts/thumbnails/<uuid>.jpg (veya .png)
```

- **Public URL tabanı:** `R2_PUBLIC_URL` (özel CDN domaini, ör. `https://cdn.canlifal.com`) → `$ {R2_PUBLIC_URL}/<key>` .
- Nesneler `Cache-Control: public, max-age=31536000, immutable` ile yazılır → CDN'de **1 yıl** ön-belleklenir, değişmez varsayılır.

10.3 Flutter için CDN kuralları

- Public asset'ler (`immutable`) **kalıcı disk cache**'e alınabilir; URL değişmedikçe yeniden indirilmez.
- İmzalı (private) URL'ler cache'lenmez; her erişimde `get-url` ile tazelenir.
- Görsel gösterimde `cached_network_image` + hediye asset'i (`immutable`) için uzun süreli disk politikası kullanın.

11. Cache Kuralları

11.1 Sunucu tarafı TTL (lib/cache.ts — in-memory)

Anahtar	TTL
platform ayarları	300 sn (5 dk)
hediye türleri (gifts:active)	600 sn (10 dk)
ödeme yöntemleri / kredi paketleri	600 sn
anasayfa kart/buton, temalar, rozetler, üyelik-ler	600 sn
falci listesi	15 sn
chat odaları / canlı yayınlar (anasayfa poll)	10 sn
kullanıcı profili	30 sn
gifts/version-check	30 sn (HTTP s-maxage=60, swr=120)

Admin güncellemesi sonrası ilgili anahtar `invalidateCache(...)` ile temizlenir → değişiklik anında yayılır.

11.2 İstemci (Flutter) cache stratejisi

- **Katalog benzeri veriler** (hediye, tema, kozmetik): sürüm tabanlı. `contentVersion` / `giftVersion` lokalde saklanır; `/api/gifts/version` ile karşılaştır, yalnız artınca `catalog?sinceVersion=` çek.
- **Config**: `/api/mobile/config` her cold start + foreground'a dönüşte (≥ 5 dk aradan sonra) çekilir.
- **Listeler (feed/yayın/oda)**: kısa TTL; `stale-while-revalidate` — önce cache göster, arka planda tazele.
- **Profil/cüzdan**: hediye/işlem sonrası **daima invalidate** (bakiye anlık doğru olmalı).

12. Offline Senaryoları

Durum	Beklenen davranış
Ağ yok, cache var	Son cache'lenen feed/katalog/profil gösterilir + "çevrimdışı" rozeti.
Ağ yok, cache yok	Boş-durum + "Bağlantı yok, tekrar dene" butonu.
Yazma isteği (hediye/mesaj) offline	Kuyruğa al ; her yazmaya lokal <code>idempotencyKey</code> (UUID) üret; ağ gelince gönder. Backend <code>idempotencyKey</code> ile çift-işlemi engeller (<code>GiftEvent</code> unique).
SSE kopuk	Otomatik yeniden bağlan (backoff); bağlanınca <code>?after=<sonTimestamp></code> ile kaçırılan olayları çek.
Token expired offline	İstek kuyrukta bekler; online olunca önce refresh, sonra kuyruk boşaltılır.

İlkeler: Parasal işlemler asla "iyimser" (optimistic) olarak kalıcı sayılmaz — sunucu `success:true` dönene kadar bakiye "beklemede" gösterilir. Salt-okunur ekranlar iyimser cache kullanabilir.

13. Retry Mekanizmaları

13.1 401 → otomatik token yenileme (interceptor)

```
// Dio interceptor mantığı (özet)
onError(err) async {
  if (err.response?.statusCode == 401 && !isRefreshCall) {
    final ok = await _refresh(); // POST /api/auth/mobile-refresh
    if (ok) return retryOriginalRequestWithNewToken();
    await logoutToLogin();      // refresh de 401 -> oturumu kapat
  }
}
```

Eşzamanlı 401'lerde **tek** refresh yapılır (mutex); diğer istekler yeni token'ı bekler.

13.2 Ağ/5xx için exponential backoff

- Yeniden denenebilir: bağlantı hatası, timeout, 502/503/504 .
- Denenmez:** 400/401(refresh sonrası)/403/404/409/422 .
- Backoff: $\min(2^n * 500ms, 8s)$ + jitter, **maks 3** deneme.
- 429 → yanıtta `Retry-After` varsa ona uy, yoksa backoff.

13.3 Idempotency (çift-işlem koruması)

- Tüm parasal/yan-etkili POST'lar (hediye gönder, satın alma) istek başına **benzersiz idempotencyKey** taşır.
- Retry aynı key ile yapılır → backend `GiftEvent.idempotencyKey` unique kontrolüyle işlemi tekrar etmez, **aynı sonucu** döndürür.

14. Background & Foreground Davranışları

14.1 Foreground'a dönüş (resume)

1. `/api/mobile/config` tazele (force-update / bakım / feature flag).
2. Access token TTL kontrolü; yakında dolacaksa proaktif refresh.
3. Açık ekranın SSE'sini yeniden bağla, `?after=<lastEventTs>` ile kaçırılanları çek.
4. Cüzdan/profil ve aktif oturumları (`/api/user/active-sessions`) tazele; aktif canlı oturum varsa devam banner'ı/ `IncomingCallModal` göster.

14.2 Background'a geçiş

- SSE bağlantılarını kapat (batarya/soket tasarrufu).
- Aktif WebRTC (sesli/görüntülü oturum) varsa platform politikasına uygun şekilde ya sürdür ya nazikçe sonlandır; oturum durumu **sunucuda** kalır.
- Zamanlayıcı/poll'leri durdur.

14.3 Push (OneSignal) — arka plandayken tek gerçek zamanlı kanal

- Backend olayları push ile bildirir: oturum isteği/kabulü (`/api/fortune-tellers/...`), hediye, takip, yayın açıldı, mesaj.
- Push payload'ında **derin bağlantı (deep link)** hedefi bulunur (ör. `canli-oda/<sessionId>` , `video/<streamId>`); açılışta ilgili ekrana yönlendir ve gerçek durumu API'den doğrula.
- Bildirim ayarları/opt-out kullanıcı tercihlerine uyar (`/api/notifications`).

14.4 Polling aralıkları (SSE yokken fallback)

- Anasayfa yayın/oda listesi: ~10 sn (sunucu cache ile uyumlu).
- Falcı listesi: ~15 sn. • Mesaj/olay: SSE tercih; SSE yoksa 3-5 sn `?after=` .

15. Acceptance Test Senaryoları

Aşağıdaki senaryolar **web ↔ mobil parite** kabul kriterleridir. Referans betikler: `fortune_telling_platform/scripts/acceptance-tests/` .

AT-1 Kimlik & Oturum

- **Verildiğinde** geçerli e-posta/şifre, **login** çağrılır → `accessToken+refreshToken+user` döner; `user.jetonBalance` web ile **aynı**.
- Access token 7 gün sonra/expired iken herhangi bir korumalı istek → 401 → otomatik refresh → istek tekrar başarılı.
- Geçersiz refresh → 401 → kullanıcı login ekranına düşer.

AT-2 Zorunlu Güncelleme / Bakım

- `minVersion` > uygulama sürümü iken cold start → engelleyici güncelleme ekranı; başka ekrana geçilemez.
- `maintenanceMode:true` → bakım ekranı; API çağrıları yapılmaz.

AT-3 Hediye Gönderimi (parasal parite)

- Kullanıcı 5 adet 100 jetonluk hediye gönderir → `jetonBalance` **500 azalır**; alıcı net/komisyon dağıtımı backend'in döndürdüğü değerlerle birebir; web'de aynı hediye aynı sonucu verir.
- Aynı `idempotencyKey` ile tekrar gönderim (retry) → **çift düşün olmaz**, aynı `giftEventId` döner.
- Yetersiz bakiye → 400/402 tarzı hata, bakiye değişmez.

AT-4 Delta Katalog Senkronizasyonu

- Lokal `giftVersion` = server → `catalog` çağrılmaz (bant tasarrufu).
- Admin yeni hediye ekleyip `contentVersion` artınca → `version` farkı algılanır → `catalog?sinceVersion=` yalnız yeni öğeyi getirir; yeni hediye hem web hem mobilde görünür.

AT-5 Canlı Oturum (real-time parite)

- Falcı oturumu kabul eder → kullanıcıya push + SSE → mobil `canli-oda/<id>` 'ye yönlendir; süre/dakika/ücret web ile aynı.
- Oturum biterken dönen `tellerEarnings`, `clientSpentTL`, `jetonTLRate` web'deki `session_ended` SSE ile **aynı**.

AT-6 Yayın / Sohbet

- Yayına hediye/mesaj → SSE `streamMessage / message` olayı hem web hem mobilde ≤ 1 sn içinde görünür.
- Beğeni (`count`) batch → `likeCount` iki platformda tutarlı artar.

AT-7 Upload & CDN

- Public avatar upload → presigned PUT (Content-Disposition dahil) → `cloud_storage_path` kaydı → public S3 URL erişilebilir; web profilinde aynı görsel görünür.
- Shorts video → R2 presigned PUT → `publicUrl` register → CDN'den `immutable` olarak servis edilir.

AT-8 Offline / Retry

- Uçak modunda hediye kuyruğa girer; ağ gelince tek seferde işlenir (idempotency ile çiftlenmez).
- 5xx/timeout'ta backoff ile en faz 3 deneme; salt-okunur ekran cache'ten dolar.

AT-9 Background/Foreground

- Uygulama arka plandan öne gelince config + cüzdan + aktif oturum tazelenir; kaçırılan SSE olayları `?after=` ile çekilir.
- Arka planda gelen push deep-link ile doğru ekrana yönlendirir ve durum API ile doğrulanır.

16. Uygulama Checklist (Flutter ekibi)

- [] `openapi.json` 'dan model üretimi (json_serializable / openapi-generator) kuruldu.
- [] `ApiResponse` / `Paginated` / `_enumFrom` ortak katmanı eklendi; bilinmeyen enum → `unknown`.
- [] Dio interceptor: Bearer ekleme + 401 tek-uçuş refresh + backoff + idempotencyKey.
- [] Cold start & resume'da `/api/mobile/config` + force-update/bakım kapıları.

- [] Sürüm tabanlı katalog cache (/api/gifts/version → delta).
- [] S3 (genel medya) + R2 (shorts) upload akışları; 100MB eşiği; public/immutable disk cache.
- [] SSE istemcisi + reconnect + ?after= boşluk doldurma; OneSignal push + deep-link.
- [] Offline kuyruk (parasal işlemler idempotency ile).
- [] Acceptance testleri AT-1...AT-9 yeşil.

Bu doküman `nextjs_space` içindeki gerçek route ve lib kaynaklarından (mobile-auth, s3, r2-storage, cache, upload, gifts, room, video-streams, mobile/config) üretilmiştir. Endpoint ve model bazında kesin sözleşme için `backend-docs/openapi.json` , `ENDPOINTS.md` , `DATABASE_REFERENCE.md` ve `schema.prisma` bağlayıcı kaynaklardır.